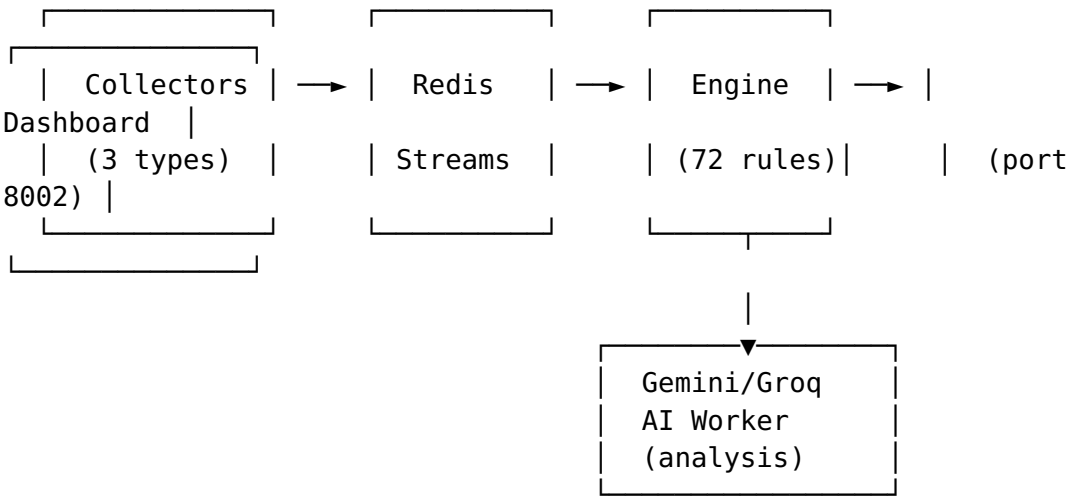
NeuralOps — AI SOC Detection Engine Guide

NeuralOps — AI SOC Detection Engine

Architecture Overview



What it does: Collects security events (web logs, network traffic, system logs), detects attacks using 72 rules, enriches with AI analysis, and displays everything in a live SOC dashboard.

Project Structure

```
neuralops/
├── collectors/          # Data collection agents
│   ├── middleware.py    # Web log collector (FastAPI plugin)
│   ├── scapy.py        # Network traffic monitor (/proc/net/tcp)
│   └── journald.py     # System log collector (journalctl)
```

```
├── config/
│   └── settings.py     # Redis, API keys, thresholds
├── engine/              # Detection engine
│   ├── rules.py        # 72 detection rules
│   ├── matchers.py     # Pattern matching logic
│   ├── scorer.py       # Confidence scoring
│   ├── dedup.py        # Alert deduplication
│   ├── feedback.py     # False positive feedback loop
│   ├── correlator.py   # Attack chain correlation
│   ├── baseline.py     # IP behavioral baselines
│   ├── enricher.py     # GeoIP enrichment
│   ├── loader.py       # Rule loader / hot-reload
│   ├── parsers.py      # Payload decode pipeline
│   ├── persistence.py  # State persistence
│   └── synthetic.py    # Synthetic event generator
├── gemini/
│   ├── dashboard.py    # SOC web dashboard (HTML+CSS+JS)
│   └── worker.py       # AI analysis worker (Gemini/Groq)
├── workers/
│   ├── detect_engine.py # Main detection engine daemon
│   ├── ml_anomaly.py   # ML anomaly detection worker
│   ├── threat_intel.py # Threat intel enrichment worker
│   ├── output.py       # Alert output adapter (file+slack)
│   └── rule_engine.py  # Rule processing engine
├── scripts/
│   ├── simulate_attacks.py  # Generate test attacks
│   ├── download_geoip.py    # Download GeoIP database
│   ├── demo.py             # Demo server
│   ├── demo_server.py      # HTTP demo server
│   ├── demo_xss.py         # XSS demo
│   ├── inject_payload.py   # Payload injection test
│   ├── seed_test_attacks.py # Seed test data
│   └── test_pipeline.py    # Pipeline integration test
├── tests/               # Unit tests
│   ├── test_rules.py
│   ├── test_parsers.py
│   └── test_scenarios.py
├── neuralops.sh         # Main launcher script
├── requirements.txt     # Python dependencies
├── docker-compose.yml   # Docker deployment
└── Dockerfile           # Container build
```

Setup (All OS)

Prerequisites

- Python 3.10+
- Redis 6+ (running locally or remotely)

- Git

1. Install Dependencies

Linux (Ubuntu/Debian):

```
sudo apt install python3 python3-pip redis-server git
sudo systemctl start redis
```

Windows: - Install Python from https://python.org - Install Redis via WSL: `wsl --install -d Ubuntu`, then inside WSL: `sudo apt install redis-server && sudo service redis start` - Or use Memurai (Windows-native Redis): https://www.memurai.com

macOS:

```
brew install python3 redis git
brew services start redis
```

2. Clone & Setup

```
git clone -b feature/sharan-soc-engine
git@github.com:TheWhitehatsclub-vit/AI-Security-Operations-
Center.git
cd AI-Security-Operations-Center
python3 -m venv venv
source venv/bin/activate    # Linux/macOS
# venv\Scripts\activate     # Windows
pip install -r requirements.txt
```

3. Configure API Key (Optional — for AI analysis)

Edit `config/settings.py` or set env var:

```
export GEMINI_API_KEY="your-key-here"    # Linux/macOS
# set GEMINI_API_KEY=your-key-here       # Windows
```

Get a free key from https://aistudio.google.com/apikey or use Groq (https://console.groq.com).

Running the System

Option A: Use the Launcher Script (Recommended)

Linux/macOS:

```
./neuralops.sh --all        # Start everything
./neuralops.sh --stop       # Stop all processes
./neuralops.sh --list       # Show running processes
./neuralops.sh --logs       # Tail all logs
./neuralops.sh --help       # See all options
```

Individual components:

```
./neuralops.sh --engine       # Detection engine only
./neuralops.sh --dashboard    # Dashboard only (port 8002)
./neuralops.sh --gemini       # AI worker only
./neuralops.sh --network      # Network collector
./neuralops.sh --system       # System log collector
```

Windows: Open Git Bash and run the same commands above.

Option B: Run Manually

```
# Terminal 1: Redis (already running as service)
redis-server

# Terminal 2: Detection Engine
python3 workers/detect_engine.py

# Terminal 3: Dashboard
python3 gemini/dashboard.py

# Terminal 4: AI Worker (optional)
python3 gemini/worker.py

# Terminal 5: Generate test attacks
python3 scripts/simulate_attacks.py
```

What Each Script Does

Collectors (Input)

Script	What it does	OS Support
		Linux/Windows/macOS

Script	What it does	OS Support
`collectors/middleware.py`	Captures HTTP requests (method, path, status, UA) by acting as a FastAPI middleware. Sends to `neuralops:logs` stream.	
`collectors/scapy.py`	Monitors `/proc/net/tcp` every 500ms for connections. Detects port scans, ARP spoofing, SYN/EST ratio anomalies. Sends to `neuralops:network` stream.	Linux only (/proc)
`collectors/journald.py`	Reads system logs via `journalctl -f`. Tracks sudo, SSH, auth failures. Sends to `neuralops:system` stream.	Linux only (journald)

Engine (Processing)

Script	What it does
`workers/detect_engine.py`	**Main engine.** Reads all 3 input streams, matches events against 72 rules, computes confidence, deduplicates, correlates attack chains, and writes alerts to `neuralops:alerts`. Includes heartbeat monitoring, rate limiting, and FP auto-tuning.
`workers/ml_anomaly.py`	ML anomaly detection using Isolation Forest. Retrains every 120s on baseline data.
`workers/threat_intel.py`	Enriches alerts with AbuseIPDB, GreyNoise, VirusTotal data.
`workers/output.py`	Writes alerts to log file and optionally sends to Slack webhook.

AI Analysis

Script	What it does
`gemini/worker.py`	Auto-consumes alerts from Redis, sends them to Gemini/Groq API for AI analysis (summary, severity reassessment, FP estimation, remediation steps), writes structured analysis to `neuralops:gemini_analysis`.
`gemini/dashboard.py`	SOC web dashboard on port 8002. Shows live alerts, AI analysis, stats, and system status. Full HTML/CSS/JS embedded.

Testing

Script	What it does
`scripts/simulate_attacks.py`	Generates 19 simulated attack events across all 3 streams (SQLi, XSS, port scans, SSH brute force, etc.). Use to test the full pipeline.
`scripts/download_geoip.py`	Downloads MaxMind GeoLite2 database for GeoIP enrichment.
`scripts/demo.py`	Simple demo server.
`scripts/demo_server.py`	HTTP server with attack endpoints.
`scripts/demo_xss.py`	XSS demonstration.
`scripts/inject_payload.py`	Manually inject a payload into a stream.
`scripts/seed_test_attacks.py`	Seed test attack data.
`scripts/test_pipeline.py`	End-to-end pipeline test.

Tests

Script	What it does
`tests/test_rules.py`	Tests all 72 detection rules.
`tests/test_parsers.py`	Tests URL decode, base64 decode, HTML entity decode.

Script	What it does
`tests/test_scenarios.py`	Integration scenario tests.

Detection Rules (72 total)

Categories: - **SQL Injection** — 10 patterns - **XSS** — 5 patterns - **Command Injection** — 8 patterns - **SSRF** — 4 patterns - **Path Traversal** — 6 patterns - **File Inclusion** — 2 patterns - **LDAP Injection** — 2 patterns - **XXE** — 2 patterns - **API Key Leak** — 3 patterns - **Network** — 18 patterns (port scans, ARP spoofing, scanner tools, etc.) - **System** — 8 patterns (SSH brute force, sudo abuse, auth failures, etc.) - **Web** — 4 patterns

Each rule includes: name, category, severity, pattern, MITRE ATT&CK TTP, kill chain stage, and base confidence score.

Dashboard Views

View	What it shows
Command Center	Alert count, active rules, killed processes, AI-triage count, recent activity feed, category pie chart, timeline
Alert Feed	Filterable alert list (by severity/source), detail panel with full alert context and AI analysis
Intelligence	Gemini/Groq AI analysis feed — summaries, severity, FP likelihood, recommended actions
Systems	Detection rules count, collector status, input streams, uptime

API Endpoints

Endpoint	Method	Description
`/api/stats`	GET	Alert statistics, counts, analysis totals
`/api/alerts?limit=N`	GET	Recent alerts (paginated)
	GET	AI analysis results

Endpoint	Method	Description
`/api/analysis?limit=N`		
`/api/stream`	GET	SSE live stream for real-time updates
`/api/feedback`	POST	Submit false positive feedback
`/health`	GET	Engine health check

Troubleshooting

Problem	Fix
Redis connection refused	Start Redis: `sudo systemctl start redis` or `redis-server`
Dashboard blank	Check port 8002 is free: `fuser -k 8002/tcp` then restart
Gemini API errors	Key quota exhausted — wait or use Groq instead
No alerts appearing	Run `scripts/simulate_attacks.py` to generate test data
Module not found	Activate venv: `source venv/bin/activate`
Permission denied (Linux)	`chmod +x neuralops.sh`

Docker Deployment

```
docker-compose up -d
```

This starts Redis + Engine + Dashboard + AI Worker in containers.